TTT4295 - Acoustic Signal Processing
Exercise 4 - Filter design and the LMS algorithm

Peter Svensson

October 29, 2026

All tasks are mandatory, except those marked "(optional)", and need to be handed in for approval (online, in Canvas). The deadline is specified in Canvas.

Tasks

1. **First-order low-pass filter**
   The "humble" first-order low-pass filter is very useful in many audio signal processing applications. An analog first-order low-pass filter has the frequency response function

$$H_{\mathrm{LPF1}}(\omega) = \frac{1}{1 + \frac{\mathrm{j}\omega}{\omega_0}} \tag{1}$$

   where $\omega_0 = 2\pi f_0$, and f_0 is the cutoff-frequency for which $|H_{\mathrm{LPF1}}(\omega)| = 1/\sqrt{2}$.

a) Your first task is to show that the filter with the frequency response function in Eq. (1) can be implemented as the digital filter with z-transform

$$H_{\mathrm{LPF1}}(z) = B\frac{1 + z^{-1}}{1 - A_1 z^{-1}} \tag{2}$$

   What are the values of B and A_1, expressed in terms of f_0 and the sampling frequency f_S?

   Hints:

   - Convert the frequency response function, $H_{\mathrm{LPF1}}(\omega)$, to a transfer function, $H_{\mathrm{LPF1}}(s)$, by setting $\mathrm{j}\omega = s$.
   - Use the bilinear transform to convert the transfer function $H_{\mathrm{LPF1}}(s)$ to a z-domain transfer function:

$$s = 2f_S\frac{z-1}{z+1}$$

b) Your second task is to plot the frequency responses of the "analog lowpass filter", i.e., Eq. (1), and of the digital lowpass filter. The digital filter's frequency response can be computed by replacing z^{-1} by $\mathrm{e}^{-\mathrm{j}\omega T} = \mathrm{e}^{-\mathrm{j}\omega/f_S}$. Do this for the sampling frequency $f_S = 48000$ Hz, and for two different cutoff-frequencies: 1000 Hz and 10 kHz. Plot the filter magnitude in dB, i.e., plot $20\lg|H(\omega)|$.

Hint: Restrict the magnitude range of the frequency response to [-30 dB,0 dB], but let the frequency range extend to $f_S/2$.

c) (optional) Your third task is to show why a first-order digital filter has an impulse response, the amplitude of which dies out exponentially, and that the first sample of the impulse response can deviate from the exponential decay. As a first step, show why a filter

$$H(z) = \frac{1}{1 - A_1 z^{-1}}$$

has an impulse response with an exponentially decreasing amplitude for all samples, provided that $|A_1| < 1$. As a second step, show why a filter

$$H(z) = \frac{1 + B_1 z^{-1}}{1 - A_1 z^{-1}}$$

will have an impulse response where the first sample deviates from the exponential decay.

Hints:

- A digital filter with the z-domain transfer function on the all-zero/non-recursive form

  $$H(z) = B_0 + B_1 z^{-1} + B_2 z^{-2} + B_3 z^{-3} + ...$$

  has an impulse response with the amplitude $[B_0, B_1, B_2, B_3, ...]$, i.e., the sequence of the B-coefficients directly tell us what the decay of the impulse response is.
- Use the Taylor expansion

  $$\frac{1}{1-x} = 1 + x + x^2 + x^3 + x^4 - ..., \qquad |x| < 1$$

  to write the filter $H(z) = \dfrac{1}{1 - A_1 z^{-1}}$ on the non-recursive form.

2. **The LMS algorithm**

Your task is to implement the LMS algorithm which can update two filter coefficients, time-step by time-step, using this algorithm:

$$\mathbf{h}_{\mathrm{new}} = \mathbf{h}_{\mathrm{old}} + \alpha \cdot e(n) \cdot \mathbf{x}_{\mathrm{mirror}}(n)$$

where $\mathbf{x}_{\text{mirror}}(n) = [x(n), x(n1)]^T$, the estimated output is $y(n) = h_0x(n)+ h_1x(n1)$, and the error is $e(n) = y_{true}(n)y(n)$. Since your implementation should handle only a filter with two coefficients, we can write out two separate equations for the two filter coefficients, h_0 and h_1:

$$h_{0,\text{new}} = h_{0,\text{old}} + \alpha \cdot e(n) \cdot x(n)$$

$$h_{1,\text{new}} = h_{1,\text{old}} + \alpha \cdot e(n) \cdot x(n-1)$$

Your implementation should:

- Define a true impulse response. Please use these fixed values:

$$\mathbf{h}_{\text{true}} = [1, 0.2]$$

- Take an input signal, $\mathbf{x}$, and the true output signal, $\mathbf{y}_{\text{true}}$ as input parameters. You should generate these two signals, and $\mathbf{x}$ could, for instance, be a zero-mean random noise signal, e.g. $\mathbf{x} = randn(1, N)$.. The true output signal should then be (using Matlab commands to exemplify)

```
ytrue =conv(x,htrue);
```

```
ytrue =ytrue(1:length(x));
```

- Take a convergence coefficient, *alpha*, as input parameter. Initially you could set this to 0.5.
- Take an initial guess of filter coefficients. You can use the values [0,0] as initial guess.
- Return the error signal as an array of values, `errorhistory`, so you can inspect it after the calculations are finished.

Your task is then to verify that your implementation works by demonstrating that the error signal has a generally decreasing trend if you plot it as `semilogy(abs(errorhistory))`.

A second task is to demonstrate that too high a convergence coefficient will make the error grow instead of decrease.